Chapter 6: Conclusion and Recommendations

6.1 Introduction

This chapter concludes the documentation of the WaitLess Digital Queue Management System by summarising the project's achievements, assessing the system against its original objectives, identifying limitations, and presenting recommendations for future work. The discussion is grounded in the actual implementation as verified through codebase analysis, automated testing, and a comprehensive 83-feature requirements audit.

6.2 Project Summary

6.2.1 Problem Addressed

Chinhoyi Provincial Hospital, like many public healthcare facilities in Zimbabwe, relied on manual queue management — physical tokens, paper registers, and verbal calls — resulting in long waits, patient anxiety, no-shows, and operational inefficiency. The WaitLess system was proposed to replace this with a real-time, web-based digital queue management platform.

6.2.2 Solution Delivered

WaitLess is a full-stack web application built with React 19 and Node.js, featuring:

- **Real-time queue management** with Server-Sent Events (SSE) for instant updates across all connected devices
- **Role-based access control** for admin, reception, triage, and clinician staff roles
- **Patient registration** with returning-patient search, QR card generation, and unique department-scoped tokens
- **Triage assessment** with emergency vital auto-classification and manual override
- **Priority-based queue ordering** (Red > Yellow > Green) with longest-wait tie-breaking
- **Queue operations** including call-next, mark missed, recall, and department transfers
- **WhatsApp Cloud API integration** with retry logic and consent-based notifications
- **Patient self-service** including self-registration, kiosk check-in, ticket tracking, and patient portal

- **Public and department-specific display boards** showing now-serving, waiting, and missed columns
- **Reporting** with average wait time, department performance, no-show, and triage reports
- **Immutable audit trail** with SHA-256 hash chain for tamper detection
- **Dual-mode storage** supporting both MySQL and file-based JSON persistence

6.2.3 Technical Architecture Summary

Layer	Technology	Key Decisions
Frontend	React 19, Vite 7, Tailwind CSS 4, Radix UI, Lucide icons	SPA with lazy-loaded routes, context-based state, custom hooks for API access
Backend	Node.js (ESM), custom HTTP router, no framework	Lightweight router with path parameters; avoids Express overhead
Real-time	Server-Sent Events (SSE)	Chosen over WebSocket for unidirectional simplicity; polling fallback
Database	MySQL (mysql2) or JSON files	Dual-mode repository pattern; file mode for development/testing
Security	bcrypt, RBAC, audit hash chain	bcrypt cost 12; session tokens; role-expansion middleware
Notifications	WhatsApp Cloud API v23.0	Retry with exponential backoff; mock provider for testing

---

6.3 Achievement of Objectives

6.3.1 Original Objectives Assessment

Objective	Status	Evidence
Replace manual queue with digital system	✅ Achieved	Full registration-to-completion workflow implemented
Real-time queue updates	✅ Achieved	SSE with < 500 ms latency; polling fallback
Reduce patient wait times	✅ Achieved (design)	Priority ordering, ETA display, tracking page
Improve patient experience	✅ Achieved	Self-registration, kiosk, tracking, WhatsApp notifications

Objective	Status	Evidence
Role-based staff access	✅ Achieved	Four roles with route-level and data-level enforcement
Triage integration	✅ Achieved	Triage assessment with emergency auto-classification
WhatsApp notifications	✅ Achieved (mock)	Cloud API integration with retry; live delivery requires production credentials
Reporting and analytics	✅ Achieved	Four report types with date filtering
Audit trail	✅ Achieved	Hash-chained, tamper-detectable, admin-only access
Offline operation	⚠️ Partial	Core operations work without internet; form auto-save and hybrid sync not implemented

6.3.2 Requirements Coverage

Based on the 83-feature requirements audit:

Status	Count	Percentage
Complete	25	30.1%
Partial	29	34.9%
Not Started	29	34.9%
Weighted Progress	—	**47.6%**

The weighted progress calculation is: `(25 complete + 0.5 × 29 partial) / 83 = 47.6%`.

This indicates that the core operational MVP is functional, but production readiness and expanded scope features remain incomplete.

6.3.3 Key Achievements

1. **Functional queue management core** — Registration, priority ordering, calls, misses, recalls, and transfers all work end-to-end
2. **Real-time architecture** — SSE provides near-instant updates; polling fallback ensures compatibility
3. **Security foundation** — bcrypt hashing, RBAC, password policy, immutable audit trail with hash chain verification
4. **Dual-mode persistence** — Repository pattern abstracts MySQL and file storage, enabling development without database setup

5. **Patient self-service** — Self-registration, kiosk check-in, ticket tracking, and patient portal reduce reception workload
6. **Automated test coverage** — 12 unit tests covering critical security and patient-handling paths, all passing
7. **Production build verified** — Frontend compiles successfully (2,618 modules); backend starts and serves all endpoints

---

6.4 Limitations

6.4.1 Technical Limitations

Limitation	Impact	Severity
In-memory session storage	All staff sessions lost on server restart; users must re-login	High
No rate limiting on auth endpoints	Susceptible to brute-force attacks	Medium
CORS configured as `*`	Allows any origin; must be restricted for production	High
No security headers (HSTS, X-Frame-Options)	Missing browser-level protections	Medium
Frontend bundle > 2 MB (raw)	Slower initial load on low-bandwidth connections	Medium
No form draft auto-save	Data lost if page refreshed during form completion	Medium
ETA uses fixed formula, not predictive	Less accurate during unusual load patterns	Low
No automated UI tests	Frontend regressions detected manually only	Medium

6.4.2 Functional Limitations

Limitation	Impact	Severity
WhatsApp delivery unverified with live credentials	Notification reliability unknown in production	High
No appointment scheduling	Patients cannot book future visits	Medium
No laboratory/radiology order management	Department-specific workflows incomplete	Medium
No pharmacy prescription workflow	Medication collection queue not managed	Medium

Limitation	Impact	Severity
No OI (Ophthalmology) confidential mode	Privacy requirements for sensitive departments not met	Medium
No audio calling on display board	Patients may miss visual-only calls in noisy environments	Medium
No feedback collection	No patient satisfaction data captured	Low
No staff shift management	Workload distribution not tracked	Low
No multi-hospital tenancy	System supports single hospital only	Low
No multi-language support	English-only interface	Low

6.4.3 Operational Limitations

Limitation	Impact	Severity
No hybrid cloud/local sync	Cannot failover between local and cloud deployment	Medium
No automated backup/restore	Data loss risk if manual backups not performed	High
No deployment documentation	Operations team lacks formal runbook	Medium
No load testing performed	System behaviour under concurrent load unknown	Medium
Usability testing not yet administered	SUS scores and user feedback not collected	Medium

---

6.5 Lessons Learned

6.5.1 Technical Lessons

1. **Custom router over Express** — Building a lightweight HTTP router (92 lines) provided full control over request processing, path parameter extraction, and middleware composition. This avoided framework overhead and deepened understanding of Node.js HTTP internals. However, it also meant missing built-in features like rate limiting, request validation, and OpenAPI documentation that frameworks provide.

2. **SSE over WebSocket** — Server-Sent Events proved sufficient for the unidirectional update pattern (server-to-client queue updates). The EventSource API is simpler than WebSocket, works through proxies, and includes automatic reconnection. The trade-off is no client-to-server streaming, which was not needed.

3. **Dual-mode repository pattern** — Abstracting storage behind a repository interface with file and MySQL implementations was valuable for development and testing. Tests run against file storage without database setup, while production uses MySQL. The pattern added minimal complexity and significant flexibility.

4. **Hash-chained audit trail** — Using SHA-256 hash chaining for audit events provides cryptographic tamper detection without external dependencies. The implementation redacts sensitive fields before storage and verifies chain integrity on every read. This was a high-value security feature with modest implementation effort.

5. **Context-based state management** — React Context with custom hooks (`useApiResource` , `useApiAction` , `useLiveRefresh`) provided clean state management without Redux. The pattern works well for the system's complexity level but may need restructuring if the application grows significantly.

6.5.2 Project Management Lessons

1. **Requirements traceability** — Maintaining an 83-feature audit matrix from the start ensured no requirements were silently dropped. Each feature was tracked with status (complete, partial, not started) and evidence.

2. **Iterative development** — Building the core queue management loop first (register → triage → call → complete) provided a demonstrable MVP early, allowing stakeholder feedback before expanding scope.

3. **Testing early** — Implementing automated tests for security-critical paths (password hashing, audit integrity, RBAC) before feature expansion caught regressions and provided confidence in the security foundation.

4. **Documentation alongside development** — Writing documentation after implementation, rather than before, ensured accuracy. However, this meant documentation lagged behind code changes, creating a maintenance burden.

6.6 Recommendations

6.6.1 Immediate Actions (Pre-Production)

These items must be completed before deploying WaitLess in a real hospital environment:

Priority	Recommendation	Rationale	Estimated Effort
P0	Restrict CORS origin to hospital LAN IP range	Security — prevents cross-origin attacks	1 hour
P0	Implement database-backed session persistence	Reliability — sessions survive server restart	1–2 days
P0	Verify WhatsApp Cloud API with live credentials	Functionality — confirms notification delivery	1 day
P0	Create deployment runbook and backup procedure	Operations — ensures recoverability	1 day
P1	Add rate limiting on `/api/auth/login`	Security — prevents brute-force attacks	4 hours
P1	Add security headers (HSTS, X-Content-Type-Options, X-Frame-Options)	Security — hardens browser protections	2 hours
P1	Conduct load testing (50+ concurrent users)	Reliability — validates performance under load	1–2 days
P1	Administer SUS questionnaire with hospital staff	Usability — collects user feedback before deployment	1 week
P2	Implement automated database backups	Reliability — prevents data loss	1 day
P2	Add automated UI tests (Playwright or Cypress)	Quality — catches frontend regressions	3–5 days

6.6.2 Short-Term Enhancements (Next 3 Months)

Priority	Recommendation	Rationale
P1	Complete appointment scheduling and follow-up reminders	Reduces no-shows; improves patient flow planning
P1	Implement laboratory/radiology order lifecycle	Completes department-specific workflows
P1	Add pharmacy prescription workflow	Manages medication collection queue
P2	Add audio calling on display board	Ensures patients hear calls in noisy environments
P2	Implement form draft auto-save	Prevents data loss during form completion

Priority	Recommendation	Rationale
P2	Add daily operational report with PDF/CSV export	Provides printable summaries for hospital administration
P2	Implement OI confidential mode	Privacy compliance for sensitive departments
P3	Add patient feedback collection	Captures satisfaction data for continuous improvement
P3	Add staff shift and workload management	Tracks staff allocation and workload distribution

6.6.3 Medium-Term Enhancements (3–6 Months)

Recommendation	Rationale
Implement configurable workflows and rules at runtime	Allows hospital administrators to adjust thresholds, departments, and routing without code changes
Add hybrid cloud/local deployment with sync	Enables failover between local server and cloud; ensures continuity during network outages
Implement automated offline/reconnect testing	Validates system behaviour during internet outages and recovery
Add code splitting and bundle optimisation	Reduces initial load time for low-bandwidth connections
Implement idle logout with client-side detection	Enhances security by logging out inactive sessions
Add CSRF protection for state-changing operations	Defends against cross-site request forgery attacks
Integrate web push notifications	Provides an additional notification channel beyond WhatsApp

6.6.4 Long-Term Vision (6–12 Months)

Recommendation	Rationale
Multi-hospital tenancy with per-facility configuration	Enables deployment across multiple hospitals with data isolation
Multi-language support (Shona, Ndebele, French)	Serves diverse patient populations in Zimbabwe and beyond

Recommendation	Rationale
AI-based waiting time prediction	Uses historical service data to predict wait times more accurately than fixed formula
AI-based bottleneck detection	Identifies congestion patterns and recommends staffing adjustments
Patient chatbot for FAQs and navigation	Reduces reception workload; guides patients through self-service
Integration with Hospital Information Systems (HIS)	Connects queue management with patient records, billing, and clinical systems
Mobile application (React Native)	Provides native mobile experience for patients and staff
FHIR-compatible API layer	Enables interoperability with standards-compliant healthcare systems

---

6.7 Future Research Directions

1. **Predictive queue modelling** — Applying machine learning to historical service-time data to predict wait times, detect anomalies, and recommend staffing adjustments. This requires sufficient clean historical data (6+ months of operation).
2. **Patient flow optimisation** — Analysing queue data to identify systemic bottlenecks, optimal department routing, and staffing patterns. Research could apply queueing theory models (M/M/c, priority queues) to the hospital's actual data.
3. **Digital health equity** — Studying the impact of digital queue management on patient outcomes across demographic groups (elderly, rural, low-literacy) to ensure the system does not exacerbate health disparities.
4. **WhatsApp API effectiveness** — Measuring the impact of WhatsApp notifications on no-show rates, patient satisfaction, and queue efficiency compared to the pre-digital baseline.
5. **Offline-first architecture patterns** — Researching robust sync mechanisms for healthcare applications that must operate in environments with intermittent internet connectivity, particularly in resource-limited settings.

6.8 Project Significance

The WaitLess Digital Queue Management System demonstrates that modern web technologies can address real healthcare operational challenges in resource-limited settings. Key contributions include:

1. **Practical solution for Zimbabwean healthcare** — Designed specifically for Chinhoyi Provincial Hospital's constraints (intermittent internet, limited IT staff, LAN deployment), the system is adaptable to similar facilities across the region.
2. **Lightweight architecture** — The custom router, SSE-based real-time, and dual-mode storage demonstrate that full-featured healthcare applications can be built without heavy frameworks, reducing complexity and maintenance burden.
3. **Security by design** — The hash-chained audit trail, bcrypt password hashing, and role-based access control establish a security foundation that can be extended as the system grows.
4. **Scalable foundation** — The modular architecture (services, repositories, controllers, contexts) supports incremental feature addition without requiring rewrites.
5. **Documentation as a first-class deliverable** — This documentation set (Chapters 1–6) provides a complete record of the system's analysis, design, implementation, testing, and evaluation, serving as both an academic submission and a technical reference for future developers.

6.9 Conclusion

The WaitLess Digital Queue Management System was conceived to replace manual queue management at Chinhoyi Provincial Hospital with a real-time, web-based platform. The system successfully delivers a functional queue management core — patient registration, triage assessment, priority-based ordering, real-time updates, WhatsApp notifications, patient self-service, display boards, reporting, and audit trail — built on a modern technology stack (React 19, Node.js, MySQL, SSE).

At the time of this documentation, the system achieves 47.6% weighted progress against 83 audited requirements, with 25 features complete and 29 partially implemented. The core operational MVP is functional and verified through 12 automated unit tests (100% pass rate), 39 manual functional test cases (100% pass rate), and successful production build verification.

The system is not yet production-ready. Critical security hardening (CORS restriction, session persistence, rate limiting), live WhatsApp verification, deployment documentation, load testing, and usability evaluation must be completed before real-patient deployment. Functional gaps in appointment scheduling, specialist department workflows, and offline continuity also remain.

Nevertheless, the architecture provides a solid, extensible foundation. The recommendations in this chapter outline a clear path from the current state to a production-ready system, followed by short-term functional enhancements and a long-term vision for multi-hospital deployment, AI-assisted optimisation, and HIS integration.

The WaitLess system demonstrates that thoughtful application of modern web technologies, combined with a deep understanding of the operational context, can produce meaningful improvements in healthcare service delivery — even in resource-limited environments.

References

1. React Documentation. (2025). *React 19*. Retrieved from https://react.dev
2. Vite Documentation. (2025). *Vite 7*. Retrieved from https://vite.dev
3. Tailwind CSS Documentation. (2025). *Tailwind CSS v4*. Retrieved from https://tailwindcss.com
4. Meta Platforms. (2025). *WhatsApp Cloud API Documentation, v23.0*. Retrieved from https://developers.facebook.com/docs/whatsapp/cloud-api
5. MDN Web Docs. (2025). *Server-Sent Events*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
6. OWASP Foundation. (2023). *OWASP Top 10*. Retrieved from https://owasp.org/www-project-top-ten/
7. Node.js Documentation. (2025). *Node.js Test Runner*. Retrieved from https://nodejs.org/api/test.html
8. Brooke, J. (1996). *SUS: A 'quick and dirty' usability scale*.
9. MySQL Documentation. (2025). *MySQL 8.0 Reference Manual*. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
10. bcrypt Documentation. (2025). *bcryptjs*. Retrieved from https://github.com/dcodeIO/bcrypt.js